Python-Grundlagen 3: Dateien, Fehler und Module

M1 · Python-Grundlagen

Version 0.10

Dateien lesen und schreiben

```
with open(path, encoding="utf-8") as f:
    content = f.read()          lesen: ganzer Inhalt als ein String
with open(path, encoding="utf-8") as f:
    lines = f.readlines()       erneut geöffnet; Liste der Zeilen, "\n" bleibt
with open(path, encoding="utf-8") as f:
    for line in f:              zeilenweise
        text = line.strip()     Leerraum an den Rändern weg
with open(path, "w", encoding="utf-8") as f:
    f.write("0.89  Gehirn\n")   "w": schreiben, alter Inhalt weg; "\n" selbst
with open(path, "a", encoding="utf-8") as f:
    f.write("0.72  Zeit\n")     "a": anhängen ans Ende
```

- `with` schließt die Datei am Ende des Blocks, auch nach einer Ausnahme. Rückgängig macht es nichts: Bei `"w"` ist der alte Inhalt weg, sobald die Datei geöffnet ist.
- Eine geöffnete Datei hat eine Leseposition, die mit jedem Lesen vorrückt. `read()` und `readlines()` liefern den Rest ab dieser Position; ein zweites `read()` im selben Block liefert `""`. Zum erneuten Lesen die Datei neu öffnen.
- `write` hängt keinen Zeilenumbruch an; `"\n"` gehört in den String.

Pfade mit pathlib

```
from pathlib import Path
Path.cwd()                          das Arbeitsverzeichnis des Kernels
p = Path("data") / "x.json"         Pfad zusammensetzen
p.exists()                          Datei oder Ordner vorhanden?
Path("out").mkdir(exist_ok=True)    anlegen; vorhandener Ordner erlaubt
open(p, encoding="utf-8")           open akzeptiert ein Path-Objekt
```

- Relative Pfade zählen vom Arbeitsverzeichnis des Kernels aus, meist dem Ordner des Notebooks, in Google Colab `/content`.
- `Path.read_text()` und `write_text()` lesen und schreiben kleine Dateien ohne `open`; `p.name`, `p.suffix`, `p.parent` zerlegen den Pfad.

JSON

```
import json
with open(path, "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2, ensure_ascii=False)
with open(path, encoding="utf-8") as f:
    data = json.load(f)
text = json.dumps(data)             Objekt -> JSON-Text (String)
data = json.loads(text)             JSON-Text (String) -> Objekt
```

Python	JSON	Zurück
`dict`	Objekt `{"a": 1}`	`dict`, Schlüssel `str`
`list`, `tuple`	Array `[1, 2]`	`list`
`str`, `int`, `float`	String, Zahl	`str`, `int`, `float`
`True`, `False`, `None`	`true`, `false`, `null`	unverändert
`ndarray`, `set`, eigene Klassen	nicht direkt: `TypeError`	

- `dump` und `load` arbeiten mit einer Datei, `dumps` und `loads` mit einem String.
- `indent=2` bricht lesbar um, `ensure_ascii=False` lässt Umlaute als Zeichen stehen.
- Eine Datei enthält genau ein JSON-Dokument; mehrere Datensätze kommen als eine Liste hinein.
- Ungültiger Text löst `json.JSONDecodeError` aus, eine Unterklasse von `ValueError`.

Module und Imports

```
import math                         math.sqrt(2): Herkunft bleibt sichtbar
from math import sqrt               nur dieser Name, ohne Präfix
import numpy as np                  Kurzname

%%writefile tools.py                eigene Zelle: 1. Zeile, darunter nur Dateiinhalt

import tools                        nächste Zelle: tools.py im Arbeitsverzeichnis
importlib.reload(tools)             nach Änderungen; import importlib
tools.__file__                      welche Datei geladen wurde
```

- Ein Modul ist eine `.py`-Datei mit eigenem Namensraum: `import json` im Notebook gilt in `tools.py` nicht.
- Python führt die Datei nur beim ersten `import` aus. Nach Änderungen `reload` oder Kernel-Neustart; `from`-Imports danach wiederholen.
- `%%writefile` überschreibt die Datei vollständig.
- Beim Import geht Python die Ordnerliste `sys.path` durch. Eigene Dateien nie `json.py` oder `numpy.py` nennen, sie verdecken die Bibliothek.
- `if __name__ == "__main__":` schützt Code, der nur beim direkten Start (`python tools.py`) laufen soll.

Hochschule Worms
University of Applied Sciences

Fehlerklassen

Alle sind Unterklassen von `Exception`. ↳ markiert eine Unterklasse der Klasse darüber; `A: B, C` nennt eine Oberklasse mit ihren Unterklassen.

Klasse	Auslöser	Beispiel
`NameError`	ein Name ist nicht definiert	Tippfehler, Zelle nicht ausgeführt
↳ `UnboundLocalError`	lokaler Name vor der Zuweisung gelesen	`count += 1` ohne vorherige lokale Zuweisung oder Parameter
`TypeError`	falscher Typ oder Aufruf passt nicht zur Signatur	`"3" + 1`, `len(5)`, unbekanntes Argument
`ValueError`	der Typ passt, der Wert nicht	`int("x")`
↳ `json.JSONDecodeError`	Text ist kein gültiges JSON	`json.load` auf eine kaputte Datei
`LookupError`: `KeyError`, `IndexError`	Schlüssel fehlt, Position außerhalb	`d["author"]`, `xs[100]`
`AttributeError`	Attribut oder Methode fehlt	`xs.push(1)`, vergessenes `self.`
`OSError`: `FileNotFoundError`	Datei oder Pfad fehlt	`open("fehlt.json")`
`ImportError`: `ModuleNotFoundError`	Name im Modul fehlt, Modul nicht in `sys.path`	`from tools import x`, `import tols`
`ZeroDivisionError`, `AssertionError`	Division durch null, `assert` schlägt fehl	`x / 0`, eine Prüfzelle meldet einen Fehler

Einen Traceback lesen

Eine Ebene je Aufruf, in Aufrufreihenfolge; die unterste Ebene ist die Stelle, an der die Ausnahme entstand. `Cell In[12], line 7` nennt Zelle und Zeile.

1. **Die letzte Zeile zuerst:** Fehlerklasse und Meldung.
2. **Dann von unten nach oben** bis zur untersten Zeile des eigenen Codes; Ebenen aus Bibliotheksdateien wie `json/decoder.py` dabei übergehen.
3. **Die Ursache liegt oft höher:** Der falsche Wert kommt meist aus einem Aufruf weiter oben.

try, except, raise

```
def load_or_empty(path):                    Datei optional, Inhalt nicht
    try:
        return load_passages(path)          kann scheitern
    except FileNotFoundError as e:          diese Klasse und Unterklassen
        print("Datei fehlt:", e)            e: das Ausnahmeobjekt
        return []
    # ValueError aus load_passages wandert zum Aufrufer

def score_label(score):
    if not 0 <= score <= 1:                 if prüft, raise löst aus
        raise ValueError(f"score out of range: {score}")

class CorpusError(Exception):               Beispiel einer eigenen Klasse
    """Die Kursdaten haben nicht das erwartete Format."""
raise CorpusError("key 'passages' missing")
```

- Entsteht im `try`-Block eine Ausnahme, wird der Rest des Blocks übersprungen und der erste passende `except` läuft. Passt keiner, wandert die Ausnahme zum Aufrufer.
- `except (A, B):` fängt mehrere Klassen. `except ValueError` fängt auch Unterklassen wie `JSONDecodeError`.
- `except Exception:` fängt `Exception` und alle ihre Unterklassen, also auch Fehler im eigenen Code wie einen Tippfehler (`NameError`). Ohne passende Behandlung oder erneutes `raise` bleiben die verborgen. Empfehlung: nur die Klassen abfangen, die an dieser Stelle behandelt werden können.
- Meldungen in `raise` sind wie der Code englisch.
- Eine eigene Fehlerklasse ist eine Unterklasse von `Exception`; sie braucht keine eigenen Methoden oder Attribute, ein Docstring als Inhalt genügt. `CorpusError` oben ist ein Beispiel, kein Teil der Standardbibliothek. Aufrufer fangen sie gezielt mit `except CorpusError` ab; von `ValueError` abgeleitet, fängt auch `except ValueError` sie noch.

assert oder raise? Prüfen oder abfangen?

`assert` oder `raise`?

- `assert` prüft Annahmen in Tests und Prüfzellen und lässt sich abschalten (`python -O`).
- Eingaben prüft die Funktion selbst: mit `if` prüfen, bei verletzter Bedingung mit `raise` eine Ausnahme auslösen.

Prüfen oder abfangen?

- Vorher prüfen (`if key in d`, `d.get(key, default)`), wenn die Prüfung einfach ist und danach nichts mehr schiefgehen kann, etwa bei einem Schlüssel in einem Dict.
- Abfangen (`try`), wenn der Zugriff den Fehler ohnehin selbst meldet oder sich zwischen Prüfung und Zugriff etwas ändern kann, etwa eine Datei, die ein anderes Programm inzwischen löscht.

else, finally und weiterreichen

```
try:
    passages = load_passages(path)
except FileNotFoundError as e:
    print("Datei fehlt:", e)
else:                                       nur ohne Ausnahme im try-Block
    print(len(passages), "Absätze")
finally:                                    immer, auch nach return im except
    print("Versuch beendet")

try:
    ...
except KeyError:
    raise                                   dieselbe Ausnahme erneut auslösen
except ValueError as e:
    raise ValueError("bad line") from e     neue Meldung, Ursache bleibt
```

- `else` läuft nur, wenn der `try`-Block ohne Ausnahme durchgelaufen ist. Dorthin gehört der Code, der das Ergebnis weiterverwendet; seine Fehler fängt das `except` darüber nicht.
- `finally` läuft immer, auch nach einem `return` im `except`. Es ist für Aufräumarbeiten; bei Dateien übernimmt das schon `with`.
- Bloßes `raise` reicht die gerade behandelte Ausnahme unverändert weiter. `raise … from e` ersetzt sie durch eine neue und behält die Ursache im Traceback.